01 Congestion Window

It is maintained by the sender and represents how much unacknowledged data TCP is currently willing to have in flight, based on its estimate of network capacity.

It can be expressed in bytes or in segments.

FOR EXAMPLE

$$cwnd = 10\ MSS$$

If:

$$MSS = 1460\ bytes$$

Then:

$$cwnd = 10 \times 1460$$

$$cwnd = 14{,}600\ bytes$$

The congestion window limits how much data can be outstanding (in flight).

Here, outstanding means transmitted but not yet acknowledged.

The sender uses cwnd to decide how aggressively to transmit.

02 ssthresh (Slow-Start Threshold)

ssthresh separates Slow Start from Congestion Avoidance.

It determines which of TCP's two growth phases is used:

- Below the threshold, TCP uses Slow Start.
- At or above the threshold, TCP uses Congestion Avoidance.

FOR EXAMPLE

$$\text{cwnd} = 8 \text{ MSS}, \text{ssthresh} = 32 \text{ MSS}$$

Because cwnd < ssthresh, TCP is in Slow Start.

If the congestion window grows to 32 MSS, TCP switches to Congestion Avoidance.

- The lecture describes setting ssthresh to approximately half the amount of data that was in flight when loss was detected.

For a simplified example:

$$\text{FlightSize} = 40 \text{ MSS}$$

Then,

$$\text{ssthresh} = \frac{40}{2} = 20 \text{ MSS}$$

Effective Window

$$\text{Effective Window} = \min(\text{cwnd}, \text{rwnd})$$

Suppose:

$$\text{cwnd} = 30 \text{ MSS}, \text{rwnd} = 20 \text{ MSS}$$

Then:

$$\min(30, 20) = 20 \text{ MSS}$$

The receiver's window is the limiting factor.

If rwnd increases to 40 MSS, the congestion window becomes the limiting factor:

$$\min(30, 40) = 30 \text{ MSS}$$

03 How is sending rate related to cwnd?

The lecture gives this approximate relationship:

$$\text{Sending Rate} \approx \frac{\text{cwnd}}{\text{RTT}}$$

Where:

- cwnd is measured in bytes.
- RTT is measured in seconds.
- The result is bytes per second.

FOR EXAMPLE

$$\text{cwnd} = 100{,}000 \text{ bytes}$$

$$\text{RTT} = 0.1 \text{ seconds}$$

Then:

$$\text{Rate} \approx \frac{100{,}000}{0.1} = 1{,}000{,}000 \text{ bytes/s}$$

Converting to bits per second:

$$1{,}000{,}000 \times 8 = 8{,}000{,}000 \text{ bits/s}$$

So the approximate rate is:

8 Mb/s

Approximation: This is an approximation. Actual throughput also depends on losses, ACK behavior, available capacity, receiver limits, and other factors.

Why does TCP increase cwnd?

At the beginning of a connection, TCP does not know the available network capacity.

If it started with a huge congestion window, it might overwhelm the network.

If it always used a tiny window, it might waste available capacity.

TCP therefore increases the window carefully, using Slow Start and Congestion Avoidance, and reduces it when congestion is detected.

This is the fundamental pattern for the rest of the lecture.

The two phases have different goals:

Phase	Goal	Growth
Slow Start	Discover available capacity quickly	Exponential
Congestion Avoidance	Probe for additional capacity carefully	Approximately linear

04 Slow Start

During Slow Start, TCP increases cwnd by approximately one MSS for each new ACK that acknowledges previously unacknowledged data.

During the next RTT, approximately 20 segments may be acknowledged.

This produces exponential growth.

EXAMPLE

Suppose initial cwnd = 10 MSS and ssthresh = 80 MSS

The window develops approximately as follows:

RTT	cwnd at the end of the growth round
Start	10 MSS
1	20 MSS
2	40 MSS
3	80 MSS

The window doubles each RTT under the simplified assumption of successful delivery and a full window of ACKs.

Once cwnd reaches ssthresh, TCP transitions to Congestion Avoidance.

Mathematically:

$$\text{cwnd}_n \approx \text{cwnd}_0 \times 2^n$$

Where:

- cwnd_0 is the initial window.
- n is the number of RTTs.

When does Slow Start stop?

In the lecture's simplified model, Slow Start stops when:

1. cwnd reaches ssthresh
2. A loss event causes TCP to change its congestion-control behavior.

05 Congestion Avoidance

Once TCP reaches ssthresh, it no longer doubles its window every RTT.

Instead, it increases the congestion window approximately linearly.

The lecture gives this per-ACK rule:

$$\text{cwnd} \mathrel{+}= \frac{\text{MSS}^2}{\text{cwnd}}$$

Here, the quantities are in bytes, and the increase is approximately MSS^2 / cwnd bytes per ACK.

CONGESTION AVOIDANCE EXAMPLE

Suppose, cwnd = 80 MSS

The congestion window grows approximately as follows:

$$80 \rightarrow 81 \rightarrow 82 \rightarrow 83 \rightarrow 84 \rightarrow \ldots$$

Each RTT adds approximately one MSS, assuming normal ACK behavior and no loss.

This is linear growth.

The exact shape depends on the congestion-control algorithm and loss events.

The general idea is:

1. Increase the window.
2. Detect congestion.
3. Reduce the window.
4. Increase again.

This repeated pattern is called the saw-tooth pattern.

Note: Suppose cwnd = 40 MSS.

If each of the 40 ACKs increases the window by approximately:

$$\frac{1}{40}\text{MSS}$$

Then the total increase over an RTT is:

$$40 \times \frac{1}{40} = 1 \text{ MSS}$$

The window grows from approximately 40 to 41 MSS.

06 Slow Start: Window and Rate by RTT

Initial window = 10 MSS, MSS = 1460 bytes, RTT = 50 ms, so let's calculate each column:

RTT 1

The window is cwnd = 10 MSS = 10 × 1460 = 14,600 bytes = 14.6 KB

$$\text{Rate} \approx \frac{\text{cwnd} \times 8}{\text{RTT}}$$

Substitute the values:

$$\text{Rate} \approx \frac{14{,}600 \times 8}{0.05}$$

$$\text{Rate} \approx 2{,}336{,}000 \text{ b/s}$$

Therefore: Rate ≈ **2.3 Mb/s**

RTT 2

The congestion window approximately doubles:

$$10 \rightarrow 20 \text{ MSS}$$

Bytes:

$$20 \times 1460 = 29{,}200 \text{ bytes}$$

Rate:

$$\frac{29{,}200 \times 8}{0.05} = 4{,}672{,}000 \text{ b/s}$$

Approximately: **4.7 Mb/s**

RTT 3

The window becomes:

$$40 \text{ MSS}$$

Bytes:

$$40 \times 1460 = 58{,}400 \text{ bytes}$$

Rate:

$$\frac{58{,}400 \times 8}{0.05} = 9{,}344{,}000 \text{ b/s}$$

Approximately: **9.3 Mb/s**

Complete table

RTT	cwnd	Window in bytes	Approximate rate at 50 ms RTT
1	10 MSS	14,600 B	2.3 Mb/s
2	20 MSS	29,200 B	4.7 Mb/s
3	40 MSS	58,400 B	9.3 Mb/s
4	80 MSS	116,800 B	18.7 Mb/s
5	160 MSS	233,600 B	37.4 Mb/s
6	320 MSS	467,200 B	74.8 Mb/s
7	640 MSS	934,400 B	149.5 Mb/s

Important limitation

These are theoretical estimates based on the congestion window and RTT.

They do not guarantee that the network will deliver data at those rates.

Actual throughput can be lower because of:

- Available link capacity.
- Receiver window limitations.
- Packet loss.
- Delayed ACKs.
- Other connections sharing the network.
- Processing and protocol overhead.

07 Downloading a 100 KB Web Page

Imagine that you want to download a 100 KB web page.

Assume the simplified lecture example:

- MSS = 1460 bytes
- Initial window = 10 MSS
- RTT = 50 ms

The first window can carry approximately:

$$10 \times 1460 = 14{,}600 \text{ bytes}$$

If the next round can send 20 MSS, it can carry:

$$20 \times 1460 = 29{,}200 \text{ bytes}$$

The cumulative amount sent after two such rounds is approximately:

$$14{,}600 + 29{,}200 = 43{,}800 \text{ bytes}$$

The third round can send 40 MSS:

$$40 \times 1460 = 58{,}400 \text{ bytes}$$

The cumulative amount is now:

$$43{,}800 + 58{,}400 = 102{,}200 \text{ bytes}$$

That is enough payload for a 100 KB object if KB means 1,000 bytes, assuming successful delivery and ignoring additional protocol and application effects.

Thus, the object can be sent over approximately three data rounds in this simplified example.

Why is this important?

For small transfers, the number of round trips can matter more than the maximum bandwidth of the link.

Even if the link supports 1 Gb/s, a short transfer may finish before TCP's window grows enough to use most of that capacity.

08 Detecting Loss

TCP needs a way to decide when a segment has been lost.

The lecture introduces two important signals:

1. Retransmission Timeout (RTO)
2. Duplicate ACKs

A timeout indicates that TCP has waited too long without receiving the expected acknowledgment.

Three duplicate ACKs indicate that later segments are still arriving, but a gap exists in the received data.

The distinction is essential because Tahoe, Reno, and NewReno react differently depending on how loss is detected.

1. What is RTO?

It is the amount of time TCP waits before deciding that a segment may need to be retransmitted because its acknowledgment has not arrived.

If the acknowledgment does not arrive before the timer expires, TCP retransmits the segment.

However, selecting the correct timeout is difficult.

If the timeout is too short, TCP may retransmit segments that were merely delayed.

If the timeout is too long, TCP may waste time waiting after a genuine loss.

That is why TCP estimates the round-trip time.

2. What is RTT?

It is the time taken for data to travel from the sender to the receiver and for an acknowledgment to return.

3. What is SRTT?

SRTT stands for Smoothed Round-Trip Time.

For the first valid RTT sample, the lecture gives:

$$SRTT = R$$

Where R is the measured RTT sample.

If the first RTT measurement is 100 ms:

$$SRTT = 100 \text{ ms}$$

For later samples, TCP uses a weighted average:

$$SRTT = (1 - \alpha)\, SRTT + \alpha\, R'$$

The lecture specifies:

$$\alpha = \frac{1}{8}$$

Here R' is the new RTT measurement.

This formula gives more weight to the previous smoothed estimate than to the newest sample.

4. What is RTTVAR?

RTTVAR stands for Round-Trip Time Variation.

It estimates how much the RTT measurements vary around the smoothed RTT.

If the measured RTT stays close to 100 ms, variation is relatively small.

If the measurements jump between 50 ms and 200 ms, variation is much larger.

For the first sample, the lecture gives:

$$RTTVAR = \frac{R}{2}$$

For subsequent samples:

$$RTTVAR = (1 - \beta)\, RTTVAR + \beta\, |\, SRTT - R'\, |$$

The lecture specifies:

$$\beta = \frac{1}{4}$$

The absolute difference:

$$|\, SRTT - R'\, |$$

measures the distance between the previous smoothed RTT and the new sample.

Important implementation detail: For later samples, the RTTVAR update uses the previous SRTT value before updating SRTT with the new measurement. The worked example below follows that order.

5. How is RTO calculated?

The lecture gives:

$$RTO = SRTT + \max(G, 4 \times RTTVAR)$$

Where:

- SRTT = smoothed RTT.
- RTTVAR = RTT variation.
- G = clock granularity.

The expression:

$$4 \times RTTVAR$$

adds a safety margin that grows when RTT measurements are variable.

6. What is exponential backoff?

If a retransmission timeout occurs, TCP increases the RTO before trying again.

The lecture gives:

$$RTO_{new} = 2 \times RTO_{old}$$

The doubling reduces the risk of repeatedly retransmitting too aggressively when the network is experiencing serious congestion.

EXAMPLE

First sample: 100 ms

Second sample: 120 ms

Step 1: First sample

The first RTT sample is:

$$R = 100 \text{ ms}$$

Therefore:

$$SRTT = 100 \text{ ms}$$

And:

$$RTTVAR = \frac{100}{2} = 50 \text{ ms}$$

Step 2: Calculate the new RTTVAR

The next RTT sample is:

$$R' = 120 \text{ ms}$$

Using the previous SRTT of 100 ms:

$$| SRTT - R' | = | 100 - 120 | = 20 \text{ ms}$$

The formula is:

$$\text{RTTVAR}_{new} = 0.75\ \text{RTTVAR}_{old} + 0.25\ |\ \text{SRTT}_{old} - R'\ |$$

Substitute:

$$\text{RTTVAR}_{new} = 0.75(50) + 0.25(20)$$

$$= 37.5 + 5$$

$$\text{RTTVAR} = \mathbf{42.5\ ms}$$

Step 3: Calculate the new SRTT

Now update SRTT:

$$\text{SRTT}_{new} = 0.875\ \text{SRTT}_{old} + 0.125\ R'$$

Substitute:

$$\text{SRTT}_{new} = 0.875(100) + 0.125(120)$$

$$= 87.5 + 15$$

$$\text{SRTT} = \mathbf{102.5\ ms}$$

Step 4: Calculate RTO

Ignoring clock granularity for this example:

$$\text{RTO} = \text{SRTT} + \max(G, 4\ \text{RTTVAR})$$

Using the lecture's displayed example:

$$\text{RTO} = 102.5 + 4(42.5)$$

$$\text{RTO} = 102.5 + 170$$

$$\text{RTO} = \mathbf{272.5\ ms}$$

This is the raw calculation before applying the relevant minimum RTO.

If the question tells you to apply a minimum of one second, the final RTO becomes:

$$\text{RTO} = \mathbf{1\ second}$$

7. Karn's Algorithm

Now consider a retransmission.

The sender transmits a segment, but no ACK arrives. It retransmits the segment.

Then an ACK arrives.

Which transmission did the ACK acknowledge?

- The original segment?
- The retransmitted copy?

Without additional information, the sender may not be able to distinguish between them.

Therefore, the RTT measurement is ambiguous.

Karn's algorithm says not to use RTT samples from retransmitted segments to update the RTT estimate.

This prevents a potentially incorrect measurement from corrupting SRTT and RTO.

Duplicate ACK

Why three duplicate ACKs?

One duplicate ACK might occur because of packet reordering.

Two duplicate ACKs still might not be enough to confidently distinguish loss from reordering.

Three duplicate ACKs trigger fast retransmit.

The sender retransmits the missing segment without waiting for the retransmission timeout.

EXAMPLE

Suppose Segment 2 is lost.

Sender	Receiver
Segment 1 →	Received · **ACK = 1001**
Segment 2 → **X**	**Lost**
Segment 3 →	Out of order · **ACK = 1001** (duplicate #1)
Segment 4 →	Out of order · **ACK = 1001** (duplicate #2)
Segment 5 →	Out of order · **ACK = 1001** (duplicate #3)

After the third duplicate ACK, the sender retransmits Segment 2.

It does not need to wait for the RTO to expire which can reduce recovery delay substantially.

- **Three duplicate ACKs:** Evidence of a missing segment while later segments are still arriving (kl duplicate aa whde mn l new segments arriving).
- **Fast Retransmit:** After receiving 3 duplicate ACKs, the sender retransmits the missing segment without waiting for a timeout.
- **Out-of-order segments:** The receiver stores the segments that arrive out of order instead of discarding them.
- **Cumulative ACK:** When the missing segment arrives, the receiver can acknowledge all consecutive bytes it has received, including previously stored segments.

Compare the two loss signals

Feature	Three duplicate ACKs	Retransmission timeout
What happens?	Later segments arrive despite a gap	Expected ACK does not arrive before the timer expires
Classic response	Fast retransmit	Timeout retransmission
How quickly can loss be detected?	Often faster	After the RTO expires
Classic congestion response	Less severe than timeout	More severe

09 Key Terms

Term	Meaning
cwnd (Congestion Window)	Controls how much data the sender can have in transit.
ssthresh (Slow Start Threshold)	Separates Slow Start from Congestion Avoidance.
MSS (Maximum Segment Size)	Maximum amount of TCP data in one segment.
RTT (Round-Trip Time)	Time for data to reach the receiver and for an acknowledgment to return.
Flight Size	Amount of data sent but not yet acknowledged.

Slow Start

- TCP increases cwnd exponentially, approximately doubling every RTT.
- When cwnd reaches ssthresh, TCP moves to Congestion Avoidance.

EXAMPLE

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \text{ MSS}$$

Key idea: Slow Start = Exponential Growth.

Congestion Avoidance

- TCP increases cwnd linearly, approximately 1 MSS per RTT.
- It normally begins when cwnd reaches ssthresh.
- Reno and NewReno return to Congestion Avoidance after Fast Recovery finishes.

EXAMPLE

$$8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow 12 \text{ MSS}$$

Key idea: Congestion Avoidance = Linear Growth.

10 Timeout vs. Three Duplicate ACKs

Timeout

A Timeout occurs when the sender does not receive the expected ACK before the retransmission timer expires.

The traditional response in Tahoe, Reno, and NewReno is:

1. Retransmit the missing data.
2. Set ssthresh to approximately half the Flight Size.
3. Reset cwnd to 1 MSS.
4. Enter Slow Start.

Three Duplicate ACKs

Three duplicate ACKs indicate that a segment is probably missing while later segments are still arriving.

The sender performs Fast Retransmit, retransmitting the missing segment without waiting for Timeout.

The response depends on the algorithm:

- **Tahoe:** Resets cwnd to 1 MSS and enters Slow Start.
- **Reno:** Enters Fast Recovery.
- **NewReno:** Enters Fast Recovery.

Key difference:

- **Timeout** → Retransmission after the timer expires.
- **Three duplicate ACKs** → Fast Retransmit without waiting for Timeout.

Fast Retransmit

Fast Retransmit retransmits a missing segment after receiving three duplicate ACKs.

EXAMPLE

1. Segment 2 is lost.
2. Segments 3, 4, and 5 arrive out of order.
3. The receiver sends three duplicate ACKs requesting the missing data.
4. The sender retransmits Segment 2.

> **Key idea:** Fast Retransmit detects likely packet loss without waiting for a Timeout.

11 Fast Recovery

Fast Recovery allows Reno and NewReno to recover from packet loss without immediately resetting cwnd to 1 MSS.

In classic Reno:

1. The sender receives three duplicate ACKs.
2. It performs Fast Retransmit.
3. It sets ssthresh to approximately half the Flight Size.
4. It sets cwnd = ssthresh + 3 MSS.
5. It enters Fast Recovery.
6. Additional duplicate ACKs may increase cwnd by 1 MSS each.
7. When an ACK confirms that recovery is complete, Reno sets cwnd = ssthresh.
8. It exits Fast Recovery and enters Congestion Avoidance.

EXAMPLE

If ssthresh = 5 MSS:

- During Fast Recovery: cwnd = 5 + 3 = 8 MSS.
- After recovery finishes: cwnd = ssthresh = 5 MSS.
- Then cwnd grows linearly during Congestion Avoidance.

Why does cwnd decrease when Reno exits Fast Recovery?

The extra 3 MSS were temporary window inflation to account for the three duplicate ACKs. When recovery finishes, Reno removes this inflation and sets cwnd to ssthresh.

> **Key idea:** Fast Recovery handles packet loss without immediately restarting from 1 MSS.

12 TCP Tahoe

After Three Duplicate ACKs

1. Performs Fast Retransmit.
2. Sets ssthresh to approximately half the Flight Size.
3. Resets cwnd to 1 MSS.
4. Enters Slow Start.

After Timeout

1. Retransmits the missing data.
2. Sets ssthresh to approximately half the Flight Size.
3. Resets cwnd to 1 MSS.
4. Enters Slow Start.

> **Key idea:** Tahoe restarts from 1 MSS after loss.

13 TCP Reno

After Three Duplicate ACKs

1. Performs Fast Retransmit.
2. Sets ssthresh to approximately half the Flight Size.
3. Sets cwnd = ssthresh + 3 MSS.
4. Enters Fast Recovery.
5. Additional duplicate ACKs may increase cwnd by 1 MSS each.
6. When recovery finishes, sets cwnd = ssthresh.
7. Exits Fast Recovery and enters Congestion Avoidance.

After Timeout

1. Retransmits the missing data.
2. Sets ssthresh to approximately half the Flight Size.
3. Resets cwnd to 1 MSS.
4. Enters Slow Start.

> **Key idea:** Reno uses Fast Recovery after three duplicate ACKs but restarts from 1 MSS after Timeout.

14 TCP NewReno

NewReno behaves similarly to Reno when entering Fast Recovery.

Its main improvement is handling multiple losses within the same window.

- **Full ACK:** Acknowledges all outstanding data covered by the recovery and ends Fast Recovery.
- **Partial ACK:** Acknowledges some data but indicates that another segment is still missing.
- NewReno stays in Fast Recovery after a Partial ACK and retransmits the next missing segment.
- When recovery finishes, it sets cwnd to ssthresh and enters Congestion Avoidance.

After Timeout: NewReno traditionally sets ssthresh to approximately half the Flight Size, resets cwnd to 1 MSS, and enters Slow Start.

> **Key idea:** NewReno handles multiple losses better using Partial ACKs.

Comparison Table

Feature	Tahoe	Reno	NewReno
After 3 duplicate ACKs	Fast Retransmit, reset cwnd to 1 MSS	Fast Retransmit and Fast Recovery	Fast Retransmit and Fast Recovery
After Timeout	Reset cwnd to 1 MSS	Reset cwnd to 1 MSS	Reset cwnd to 1 MSS
Fast Recovery	No	Yes	Yes
Multiple losses in one window	Inefficient recovery	May need a Timeout	Better recovery using Partial ACKs
After recovery	Slow Start after classic duplicate-ACK loss response	cwnd = ssthresh, then Congestion Avoidance	cwnd = ssthresh, then Congestion Avoidance

15 Final Memory Tricks

Concept	Trick
Slow Start:	Exponential growth.
Congestion Avoidance:	Linear growth.
Fast Retransmit:	Retransmits after three duplicate ACKs.
Fast Recovery:	Recovers from loss without immediately resetting cwnd to 1 MSS.
Tahoe:	Retransmit → Reset → Restart.
Reno:	Fast Recovery for a single loss.
NewReno:	Better recovery from multiple losses using Partial ACKs.
Timeout:	cwnd = 1 MSS in the traditional Tahoe, Reno, and NewReno algorithms.
Three Duplicate ACKs:	Tahoe resets cwnd; Reno and NewReno enter Fast Recovery.
Reno exits Fast Recovery:	cwnd = ssthresh, then Congestion Avoidance.

SACK (Selective Acknowledgment)

Cumulative ACK identifies the next expected byte, while SACK also informs the sender about later byte ranges that have been received successfully. This helps the sender retransmit missing data without unnecessarily retransmitting received data.

Scoreboard

The sender maintains a record of received, missing, and outstanding data. It uses this information to identify lost segments and prioritize retransmissions.

Pipe Algorithm

Pipe estimates how much data is still in the network during recovery. If Pipe < cwnd, the sender may transmit eligible data without exceeding the congestion window.

RACK

RACK uses acknowledgment timing information to help detect lost segments, including when later segments have already arrived.

TLP (Tail Loss Probe)

TLP sends a probe to help detect losses near the end of a transmission, when there may not be enough later segments to generate duplicate ACKs.

Concept	Memory Trick
SACK	Identifies received byte ranges.
Scoreboard	Tracks data status.
Pipe	Estimates data still in the network.
RACK	Uses timing to detect loss.
TLP	Probes for tail loss.

16 AIMD and Fairness

AIMD = Additive Increase, Multiplicative Decrease.

- **Additive Increase:** TCP gradually increases cwnd to use available network capacity.
- **Multiplicative Decrease:** When congestion is detected, TCP reduces cwnd (approximately by half in classic Reno after 3 duplicate ACKs).
- Repeating both actions creates a saw-tooth pattern.

Fairness means sharing bandwidth equally between competing flows.

- **Additive Increase:** Both flows increase by the same amount, so their absolute difference stays the same.
- **Multiplicative Decrease:** Both flows decrease by the same factor, reducing their absolute difference.

Fairness Line:

$$x = y$$

It represents equal bandwidth for both flows. AIMD can help flows move toward fair sharing, but it does not guarantee perfect fairness in every situation.

Why is fairness useful? It prevents one flow from taking a much larger share of the bandwidth than another.

RTT Unfairness

RTT = Round-Trip Time.

A shorter-RTT connection completes more round trips per second and may increase its congestion window faster in real time, giving it a bandwidth advantage.

17 Why Is TCP Not Always Fair?

1. **Multiple connections:** An application using many TCP connections may obtain more total bandwidth.
2. **RTT unfairness:** Shorter-RTT connections may get more bandwidth.
3. **Uncontrolled UDP:** UDP without congestion control can compete unfairly with TCP.
4. **Bufferbloat:** Large queues can cause high delays.

Remember: AIMD helps TCP flows move toward fair bandwidth sharing, but fairness does not necessarily mean maximum network utilization.

Throughput Model

$$\text{Throughput} \approx \frac{\text{MSS}}{\text{RTT}} \times \frac{1.22}{\sqrt{p}}$$

Where:

- MSS = Maximum Segment Size, in bytes.
- RTT = Round-Trip Time, in seconds.
- p = Packet loss probability, expressed as a fraction.
- 1.22 = Approximate constant used by the model.

The formula is an approximation, not an exact prediction of every real TCP connection. If MSS increases while other assumptions remain constant, the estimated throughput increases. However, if RTT or packet loss probability increases, the estimated throughput decreases. Long-distance TCP transfers can be challenging because they often have a high RTT and a large Bandwidth-Delay Product (BDP). Classic TCP Reno, a loss-based congestion-control algorithm, may struggle to achieve very high throughput on these paths unless packet loss is extremely low.